Funciones de ventana y programación SQL

`OVER`, funciones definidas por el usuario, variables e `IF`

Semana 13 - Sesión 25

Rel Guzman



Logro de la sesión

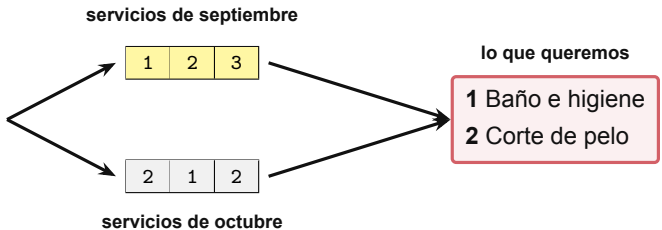
Al finalizar, podrás hacer cálculos sobre grupos de filas **sin juntarlas**, con **funciones de ventana**, y crear **tus propias funciones** con **variables** e **IF**, en MySQL y SQL Server.

Revisión: la tabla y lo que queremos

¿Qué servicios se pidieron en **septiembre** y **también** en **octubre**?

cita

id	fecha_hora	servicio_id
1	2026-09-08 10:00:00	1
2	2026-09-15 11:00:00	2
3	2026-09-22 09:00:00	3
4	2026-10-06 10:00:00	2
5	2026-10-13 11:00:00	1
6	2026-10-20 09:00:00	2

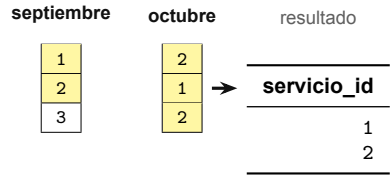


Lo que buscamos

Los servicios que están en **las dos** listas: el **1** y el **2**. El **3** (Corte de uñas) no, porque solo se pidió en septiembre.

Revisión: ahora con INTERSECT

```
SELECT servicio_id FROM cita
WHERE MONTH(fecha_hora) = 9
INTERSECT
SELECT servicio_id FROM cita
WHERE MONTH(fecha_hora) = 10
ORDER BY servicio_id;
```



El servicio **1** (Baño e higiene) y el **2** (Corte de pelo) están en las dos listas. El **3** (Corte de uñas) solo se pidió en septiembre.

INTERSECT

Cada `SELECT` arma **una lista**: la de septiembre y la de octubre. `INTERSECT` deja solo lo que está **en las dos**.

Revisión: ¿y con JOIN?

JOIN de cita consigo misma

```sql
SELECT DISTINCT c1.servicio_id
FROM cita c1
JOIN cita c2 ON c2.servicio_id = c1.servicio_id
WHERE MONTH(c1.fecha_hora) = 9
  AND MONTH(c2.fecha_hora) = 10
ORDER BY c1.servicio_id;
```

resultado

servicio_id
1
2

Sí sale, pero hay que unir `cita` consigo misma, con dos alias, y quitar los repetidos con `DISTINCT`: sin él, el 2 sale dos veces.

Revisión: una pregunta que necesita subconsulta

¿Qué pagos están **por encima del promedio**?

Así no se puede

```
SELECT id, monto FROM pago
WHERE monto > AVG(monto);

ERROR 1111 (HY000):
Invalid use of group function
```

`WHERE` revisa **una fila a la vez**, pero el promedio necesita **todas** las filas.

Con subconsulta

```
SELECT id, monto, metodo FROM pago
WHERE monto > (SELECT AVG(monto) FROM pago)
ORDER BY id;
```

resultado

id	monto	metodo
2	55.00	tarjeta
4	55.00	efectivo
8	55.00	transferencia

¿Por qué una subconsulta?

La de adentro calcula **primero** el promedio (`37.500000`) y después cada fila se compara con ese número.

La peluquería en MySQL y SQL Server

En cada gestor ejecuta **dos scripts, en orden**. Están en `ejemplos/practicas/`.

Gestor	1. Modelo físico (tablas)	2. Registros (datos)
MySQL (Workbench)	`mysql/practica_1_modelo_fisico.sql`	`mysql/practica_1_registros.sql`
SQL Server (SSMS)	`sqlserver/practica_1_modelo_fisico.sql`	`sqlserver/practica_1_registros.sql`

El primero crea la base `peluqueria` y sus tablas. El segundo carga los servicios, empleados, clientes, mascotas, citas y pagos.

Comprueba en cada gestor: `SELECT COUNT(*) FROM cita;` debe dar **10**.

Ojo con esto

El script de registros **borra** todos los datos de la base y los vuelve a cargar. Así todos vemos los mismos resultados.

Funciones de ventana

¿Para qué sirve el promedio de los pagos?

¿Cuánto deja, en promedio, cada visita a la peluquería?

```sql
SELECT AVG(monto) AS promedio
FROM pago;
```

tabla pago

id	monto	metodo	cita_id
1	35.00	efectivo	1
2	55.00	tarjeta	2
3	15.00	efectivo	3
4	55.00	efectivo	4
5	35.00	tarjeta	5
6	15.00	efectivo	8
7	35.00	tarjeta	9
8	55.00	transferencia	10

AVG →

resultado

promedio
37.500000

Ojo con esto

`AVG` **junta** las 8 filas en una. Tengo el promedio, pero ya no veo cada pago.

Quiero cada pago y, al lado, el promedio

Así se vería el reporte que queremos:

Reporte de pagos

Peluquería canina

Pago	Monto	Promedio	
1	S/ 35.00	S/ 37.50	
2	S/ 55.00	S/ 37.50	▲ más que el promedio
3	S/ 15.00	S/ 37.50	
4	S/ 55.00	S/ 37.50	▲ más que el promedio
5	S/ 35.00	S/ 37.50	
6	S/ 15.00	S/ 37.50	
7	S/ 35.00	S/ 37.50	
8	S/ 55.00	S/ 37.50	▲ más que el promedio

Lo que buscamos

Una fila **por cada pago**, y en todas el **mismo promedio**, para comparar de un vistazo.

Quiero cada pago y, al lado, el promedio

Para comparar, lo primero que se nos ocurre es pedir las columnas y el promedio juntos:

```sql
SELECT id, monto, AVG(monto) AS promedio
FROM pago;

-- MySQL: ERROR 1140: In aggregated query without GROUP BY, expression #1
--   of SELECT list contains nonaggregated column 'peluqueria.pago.id'…
-- SQL Server: Msg 8120: Column 'pago.id' is invalid in the select list because
--   it is not contained in either an aggregate function or the GROUP BY clause.
```



Por qué falla

`AVG` deja **una sola fila**, pero `id` y `monto` tienen **ocho** valores: el gestor no sabe cuál poner. Necesito el promedio **sin juntar** las filas.

`OVER`: el promedio en cada fila

```sql
SELECT id, monto, AVG(monto) OVER () AS promedio
FROM pago
ORDER BY id;
```

Se lee: «Muestra el `id` y el `monto` de cada pago y, al lado, el **promedio** de `monto` calculado sobre **todas las filas**, sin juntarlas».

resultado

id	monto	promedio
1	35.00	37.500000
2	55.00	37.500000
3	15.00	37.500000
4	55.00	37.500000
5	35.00	37.500000
6	15.00	37.500000
7	35.00	37.500000
8	55.00	37.500000

`OVER ()` dice **sobre qué filas** se calcula: con los paréntesis vacíos, sobre **las 8**.

El resultado, 37.5, se escribe en **cada fila**. Así se ve que los pagos 2, 4 y 8 (cortes de pelo) están **por encima del promedio**.

Función de ventana

Calcula sobre un grupo de filas, **la ventana**, y deja cada fila en su lugar. Se reconoce por `OVER`.

Las piezas de una función de ventana

La consulta anterior, pieza por pieza:

`AVG(monto)`	◀ ¿qué calculas? `AVG`, `SUM`, `COUNT`…
`OVER ()`	◀ sin juntar las filas, sobre **toda la tabla**

Se lee

«El promedio de `monto`, calculado sobre todas las filas, escrito en cada fila».

Las piezas de una función de ventana: dos piezas más

Dentro de los paréntesis de `OVER` puedes agregar dos piezas, las dos opcionales:

```
SUM(monto)
OVER (
   PARTITION BY metodo
   ORDER BY id
)
```

- ¿qué calculas?
- sin juntar las filas
- ¿por grupos? un cálculo por cada método de pago
- ¿en qué orden? va sumando lo anterior
- se cierra el paréntesis

`OVER ()`
toda la tabla

`OVER (PARTITION BY …)`
cada grupo por separado

`OVER (ORDER BY …)`
fila por fila, sumando lo anterior

PARTITION BY: un cálculo por grupo

¿Cuánto trabajo tiene cada empleado? Cada cita y, al lado, cuántas citas tiene **su** empleado.

```
SELECT id, empleado_id,
  COUNT(*) OVER (PARTITION BY empleado_id)
    AS citas_del_empleado
FROM cita
ORDER BY empleado_id, id;
```

empleado 1: 6 citas

empleado 2: 4 citas

resultado

id	empleado_id	citas_del_empleado
1	1	6
3	1	6
4	1	6
6	1	6
8	1	6
10	1	6
2	2	4
5	2	4
7	2	4
9	2	4

PARTITION BY

Separa las filas en grupos, como `GROUP BY`, pero **sin juntarlas**. `COUNT` cuenta dentro de cada grupo y escribe el resultado en todas sus filas.

`ORDER BY` dentro de `OVER`: el total acumulado

¿Cuánto llevamos cobrado **hoy** (4 de noviembre) después de cada pago? La fecha está en `cita`.

```sql
SELECT c.fecha_hora, p.monto,
  SUM(p.monto) OVER (ORDER BY c.fecha_hora) AS acumulado
FROM pago p JOIN cita c ON c.id = p.cita_id
WHERE CAST(c.fecha_hora AS DATE) = '2026-11-04'
ORDER BY c.fecha_hora;
```

resultado

fecha_hora	monto	acumulado
2026-11-04 09:00:00	15.00	15.00
2026-11-04 10:00:00	35.00	50.00
2026-11-04 11:00:00	55.00	105.00

cómo se calcula

lo anterior + este pago
15
15 + 35
50 + 55

SQL Server muestra la fecha como `2026-11-04 09:00:00.0000000`.

Idea clave

Con `ORDER BY`, la ventana de cada fila va **desde el primer pago del día hasta ella**. Al final del día, la caja tiene **105** soles.

Numerar: ROW_NUMBER

Ordena los pagos del más alto al más bajo y **numéralos**.

```
SELECT id, monto,
   ROW_NUMBER() OVER (ORDER BY monto DESC, id) AS puesto
FROM pago ORDER BY puesto;
```

resultado

id	monto	puesto
2	55.00	1
4	55.00	2
8	55.00	3
1	35.00	4
5	35.00	5
7	35.00	6
3	15.00	7
6	15.00	8

Los pagos 2, 4 y 8 **empatan** en 55, pero cada uno recibe **su propio número**: 1, 2 y 3.

El `id` al final del `ORDER BY` decide quién va primero en un empate. Sin él, MySQL y SQL Server los numeran distinto.

ROW_NUMBER

Numera 1, 2, 3, 4, 5… **sin repetir**, aunque haya empates.

Rankear con empates: RANK

Del más alto al más bajo, ¿en qué **puesto** queda cada pago?

```
SELECT id, monto,
   RANK() OVER (ORDER BY monto DESC) AS puesto
FROM pago ORDER BY puesto, id;
```

resultado

id	monto	puesto
2	55.00	1
4	55.00	1
8	55.00	1
1	35.00	4
5	35.00	4
7	35.00	4
3	15.00	7
6	15.00	7

Los tres de 55 comparten el puesto **1**.

El siguiente es el **4**, no el 2: hay tres pagos delante. Los puestos 2 y 3 **se saltan**.

RANK

Los empatados **comparten puesto** y después **salta**: 1, 1, 1, 4, 4, 4, 7, 7.

Rankear sin saltos: DENSE_RANK

Del más alto al más bajo, ¿en qué **puesto** queda cada pago?

```sql
SELECT id, monto,
   DENSE_RANK() OVER (ORDER BY monto DESC) AS puesto
FROM pago ORDER BY puesto, id;
```

resultado

id	monto	puesto
2	55.00	1
4	55.00	1
8	55.00	1
1	35.00	2
5	35.00	2
7	35.00	2
3	15.00	3
6	15.00	3

Es la misma consulta, cambiando `RANK` por `DENSE_RANK`.

Los tres de 55 siguen en el puesto **1**, pero los de 35 ahora quedan en el **2**, no en el 4, y los de 15 en el **3**: **no se salta** ningún número.

DENSE_RANK

Los empatados **comparten puesto** y el siguiente **no salta**: 1, 1, 1, 2, 2, 2, 3, 3.

El pago anterior: LAG

Junto a cada pago, ¿de cuánto fue el pago **anterior**?

```sql
SELECT id, monto,
   LAG(monto) OVER (ORDER BY id) AS anterior
FROM pago ORDER BY id;
```

resultado

id	monto	anterior
1	35.00	NULL
2	55.00	35.00
3	15.00	55.00
4	55.00	15.00
5	35.00	55.00
6	15.00	35.00
7	35.00	15.00
8	55.00	35.00

`LAG` trae el monto de la fila de **arriba**: el 35 del pago 1 aparece al lado del pago 2.

El pago 1 no tiene anterior: sale `NULL`.

Sirve para comparar: el pago 3 (15) fue menor que el 2 (55).

LAG

Trae un valor de la fila **anterior**, según el orden que se pone en `OVER`.

El pago siguiente: LEAD

Junto a cada pago, ¿de cuánto fue el pago **siguiente**?

```
SELECT id, monto,
   LEAD(monto) OVER (ORDER BY id) AS siguiente
FROM pago ORDER BY id;
```

resultado

id	monto	siguiente
1	35.00	55.00
2	55.00	15.00
3	15.00	55.00
4	55.00	35.00
5	35.00	15.00
6	15.00	35.00
7	35.00	55.00
8	55.00	NULL

Es la misma consulta, cambiando LAG por LEAD.

LEAD trae el monto de la fila de **abajo**: el 55 del pago 2 aparece al lado del pago 1.

El pago 8 es el último: no tiene siguiente y sale NULL.

LEAD

Trae un valor de la fila **siguiente**. Es lo contrario de LAG.

Funciones de ventana en MySQL y SQL Server

Abre y ejecuta `practica_2_ventana.sql`. Está en `ejemplos/practicas/`:

1. **MySQL:** `mysql/practica_2_ventana.sql` en Workbench.
2. **SQL Server:** `sqlserver/practica_2_ventana.sql` en SSMS.

¿Cuánto llevamos cobrado **hoy** después de cada pago?

```sql
SELECT c.fecha_hora, p.monto,
  SUM(p.monto) OVER (ORDER BY c.fecha_hora)
    AS acumulado
FROM pago p JOIN cita c ON c.id = p.cita_id
WHERE CAST(c.fecha_hora AS DATE) = '2026-11-04'
ORDER BY c.fecha_hora;
```

resultado

fecha_hora	monto	acumulado
2026-11-04 09:00:00	15.00	15.00
2026-11-04 10:00:00	35.00	50.00
2026-11-04 11:00:00	55.00	105.00

SQL Server: `2026-11-04 09:00:00.0000000`.

Idea clave

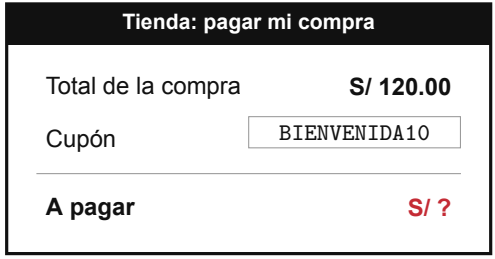
Las funciones de ventana se escriben **igual en los dos** gestores y dan los mismos valores; solo cambia cómo se muestra la fecha.

Funciones definidas por el usuario

Caso de estudio: sistema de cupones de descuento

Una tienda en línea lanza **códigos de descuento**.

- El **responsable de promociones** registra los cupones y los activa o desactiva.
- El **cliente** escribe el código al pagar.
- Un cupón descuenta un **porcentaje** o un **monto fijo** en soles.
- Tiene fecha de inicio y, si se quiere, fecha de fin.

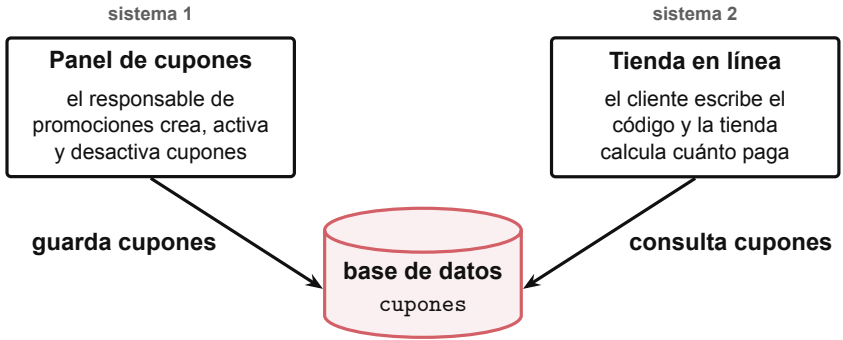


cupon

id	codigo	tipo_descuento	valor_descuento	estado	vigente_desde	vigente_hasta
1	BIENVENIDA10	porcentaje	10.00	activo	2026-01-01	NULL
2	NOVIEMBRE20	monto_fijo	20.00	activo	2026-11-01	2026-12-01
3	VERANO15	porcentaje	15.00	activo	2026-01-01	2026-04-01
4	NAVIDAD25	porcentaje	25.00	activo	2026-12-15	2026-12-26
5	PRUEBA50	monto_fijo	50.00	inactivo	2026-10-01	NULL

Dos sistemas, una misma base de datos

El caso tiene **dos programas distintos** que trabajan con la **misma base** `cupones`:



Idea clave

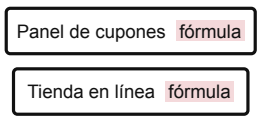
Cada sistema funciona **por su cuenta**: uno **guarda** los cupones y el otro los **usa**. Lo único que comparten es la base.

La pregunta del caso: ¿cuánto paga el cliente?

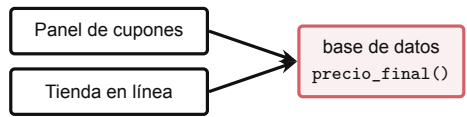
Una compra de **S/ 120**. El cálculo depende del tipo de cupón:

Cupón	Tipo	Cálculo	Paga
`BIENVENIDA10`	porcentaje: 10	$120 - 120 \times 10/100$	**S/ 108.00**
`NOVIEMBRE20`	monto fijo: 20	$120 - 20$	**S/ 100.00**

¿Dónde se escribe ese cálculo? Lo necesitan **los dos sistemas**: el panel, para mostrar cuánto quedaría una compra, y la tienda, para cobrar.



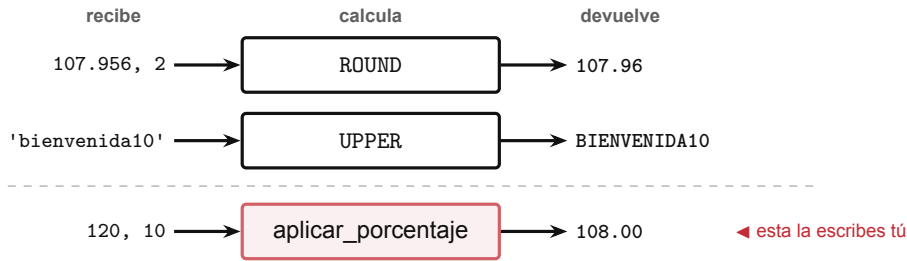
Cada sistema con **su** fórmula: si una está mal, el panel muestra un precio y la tienda cobra otro.



La fórmula se escribe **una vez**, en la base, y todos la usan.

Ya usas funciones: ahora creas las tuyas

Una función **recibe** datos, hace un cálculo y **devuelve** un valor. Ya conoces las que trae el gestor:



Función definida por el usuario

Una función que **tú escribes** con `CREATE FUNCTION`. Queda **guardada en la base de datos** y se usa dentro de un `SELECT`, igual que `ROUND` o `UPPER`.

Tu primera función en MySQL

Empecemos por el caso más simple: un cupón de **porcentaje**.

```
CREATE FUNCTION aplicar_porcentaje(
  p_precio DECIMAL(10,2),
  p_porcentaje DECIMAL(5,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
RETURN p_precio - p_precio * p_porcentaje / 100;
```

- el nombre de tu función
- lo que recibe: **parámetros** con su tipo
- … separados por comas
- el tipo de lo que devuelve
- mismos datos de entrada, mismo resultado
- el cálculo que devuelve

Se usa como cualquier función:

```
SELECT aplicar_porcentaje(120, 10) AS a_pagar;
```

resultado
a_pagar
108.00

`p_` al inicio es una costumbre: así se ve que `p_precio` es un **parámetro** y no una columna.

La misma función en SQL Server

MySQL

```
CREATE FUNCTION aplicar_porcentaje(
   p_precio DECIMAL(10,2),
   p_porcentaje DECIMAL(5,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
RETURN p_precio - p_precio * p_porcentaje / 100;

SELECT aplicar_porcentaje(120, 10);
```

SQL Server

```
CREATE OR ALTER FUNCTION dbo.aplicar_porcentaje(
   @precio DECIMAL(10,2),
   @porcentaje DECIMAL(5,2))
RETURNS DECIMAL(10,2)
AS
BEGIN
  RETURN @precio - @precio * @porcentaje / 100;
END;
GO

SELECT dbo.aplicar_porcentaje(120, 10);
```

	MySQL	SQL Server
Parámetros	`p_precio`	`@precio`: la `@` es obligatoria
Cuerpo	`RETURN …;`	entre `AS BEGIN` y `END`
Si ya existe	`CREATE` da error	`CREATE OR ALTER` la reemplaza
Para usarla	`aplicar_porcentaje(…)`	`dbo.aplicar_porcentaje(…)`

Usarla sobre las filas de una tabla

En el panel: con una compra de S/ 120, ¿cuánto pagaría el cliente con cada cupón de porcentaje?

```sql
SELECT codigo, valor_descuento,
   aplicar_porcentaje(120, valor_descuento)
     AS a_pagar
FROM cupon
WHERE tipo_descuento = 'porcentaje'
ORDER BY id;
```

resultado

codigo	valor_descuento	a_pagar
BIENVENIDA10	10.00	108.00
VERANO15	15.00	102.00
NAVIDAD25	25.00	90.00

Idea clave

La función se ejecuta **una vez por cada fila**: cada vez recibe el `valor_descuento` de esa fila. En SQL Server se escribe `dbo.aplicar_porcentaje`.

MySQL: sin `DETERMINISTIC`

```
ERROR 1418 (HY000): This function has none of
DETERMINISTIC, NO SQL, or READS SQL DATA in its
declaration…
```

SQL Server: sin `dbo.`

```
Msg 195: 'aplicar_porcentaje' is not a recognized
built-in function name.
```

La base de cupones y tu primera función

1. Crea la base `cupones`: dos scripts, en orden. Están en `ejemplos/practicas/`:

Gestor	1. Modelo físico (tablas)	2. Registros (datos)
MySQL (Workbench)	`mysql/practica_3_modelo_fisico.sql`	`mysql/practica_3_registros.sql`
SQL Server (SSMS)	`sqlserver/practica_3_modelo_fisico.sql`	`sqlserver/practica_3_registros.sql`

2. Crea la función y pruébala. Los scripts están en `ejemplos/2_cupones/`:
   - **MySQL:** `1_funciones_mysql.sql` **SQL Server:** `1_funciones_sqlserver.sql`
3. Comprueba:

```
SELECT aplicar_porcentaje(120, 10);  -- MySQL
SELECT dbo.aplicar_porcentaje(120, 10); -- SQL Server
```

resultado

a_pagar
108.00

Variables y condicional IF

Un cupón de monto fijo no es un porcentaje

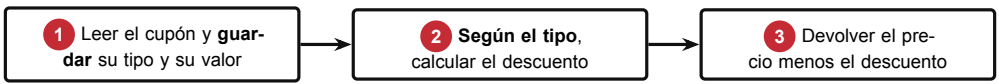
`NOVIEMBRE20` descuenta **S/ 20**, no el 20 %. Con la función de antes, el cálculo sale mal:

```
SELECT aplicar_porcentaje(120, 20);
```

resultado

`96.00` debía ser **100.00**

La función `precio_final(precio, cupon)` tiene que hacer **tres pasos**:



Variables: guardar un valor para usarlo después

Variable

Un **nombre** que guarda **un valor** mientras se ejecuta tu código. Se le da un valor con `SET`.

MySQL

```
SET @precio = 120;
SET @descuento = 20;
SELECT @precio - @descuento AS a_pagar;
-- 100
```

Guardar lo que devuelve un `SELECT`:

```
SELECT valor_descuento INTO @descuento
FROM cupon WHERE codigo = 'NOVIEMBRE20';
```

SQL Server

```
DECLARE @precio DECIMAL(10,2) = 120;
DECLARE @descuento DECIMAL(10,2) = 20;
SELECT @precio - @descuento AS a_pagar;
-- 100.00
```

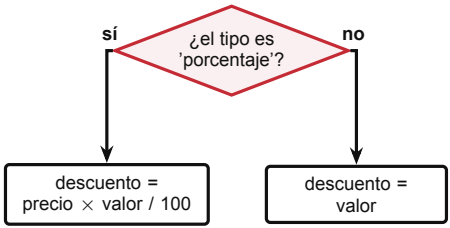
Guardar lo que devuelve un `SELECT`:

```
SELECT @descuento = valor_descuento
FROM cupon WHERE codigo = 'NOVIEMBRE20';
```

Ojo con esto

Dentro de una función de MySQL, las variables se crean con `DECLARE` y van **sin** `@`: `DECLARE v_tipo VARCHAR(10);`

IF: elegir un camino



Se ejecuta **solo uno** de los dos caminos.

MySQL

```
IF v_tipo = 'porcentaje' THEN
  SET v_descuento = p_precio * v_valor / 100;
ELSE
  SET v_descuento = v_valor;
END IF;
```

SQL Server

```
IF @tipo = 'porcentaje'
  SET @descuento = @precio * @valor / 100;
ELSE
  SET @descuento = @valor;
```

En palabras simples

En SQL Server no hay `THEN` ni `END IF`. Si un camino tiene **varias** instrucciones, van entre `BEGIN` y `END`.

Preguntas



Cierre

Función de ventana	Calcula sobre un grupo de filas sin juntarlas. Se reconoce por `OVER`.
`PARTITION BY`	Separa las filas en grupos; el cálculo se hace en cada grupo.
`ORDER BY` **en** `OVER`	Calcula en orden, sumando lo anterior: el total acumulado.
`ROW_NUMBER`, `RANK`	Numeran filas. Con empates, `RANK` salta y `DENSE_RANK` no.
`LAG`, `LEAD`	Traen un valor de la fila anterior o de la siguiente.
Función del usuario	Tu propio cálculo, guardado en la base con `CREATE FUNCTION`.
Variable	Un nombre que guarda un valor: `DECLARE`, `SET`, `SELECT … INTO`.
`IF`	Elige un camino. MySQL: `ELSEIF` y `END IF`; SQL Server: `ELSE IF`.

Referencias

- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2014). *Fundamentos de bases de datos* (6.ª ed.). McGraw-Hill.
- Oracle. MySQL Reference Manual: *Window Functions*.
- Oracle. MySQL Reference Manual: *CREATE PROCEDURE and CREATE FUNCTION* y *IF Statement*.
- Microsoft. Transact-SQL: *CREATE FUNCTION* y *IF...ELSE*.



UTP Universidad Tecnológica del Perú